Ain Shams University
Faculty of Engineering

Cinema Ticket System

CSE244: Database Systems Design

Submitted to

Dr. Nivin Atef ***TA: Eng. Esraa karam*** ***Team 5***

Mariam Mohey Ibrahiem Arafa	*24P0392*
Eithar Diaa Amin Abd Al-Aziz	*24P0383*
Doaa Shaker Mohamed Aziz Awad Elhofy	*24P0445*
Rana Salah Mohammed Fawzy	*24P0389*
Seif Shehta Abdelfattah Abdelfattah Zayed	*24P0190*
Youssef Mohamed Wafaey Abouelseoud	*24P0177*
Youssef Shehta Abdelfattah Abdelfattah Zayed	*24P0191*

Table of Contents

1. Scenario Description

The Cinema Ticket System is a database-driven application designed to manage the complete lifecycle of movie screenings and customer bookings at a multi-hall cinema complex. The system enables cinema operators to manage their halls, schedule movie shows, assign seating, and process customer bookings while giving customers a reliable record of their reservations.

The system manages the following core operational areas:

- Hall and seat management: The cinema has multiple halls, each with a defined seating capacity. Every seat belongs to a specific hall and carries attributes such as its row number, seat number, type (e.g. standard, VIP), and current availability status.
- Movie catalogue: The system maintains a library of movies, each identified by a unique Movie ID and described by its name, duration, age rating, and classification.
- Show scheduling: A show represents a single screening of a movie in a specific hall at a specific date and start time. Shows are priced individually and linked to both the movie being screened and the hall in which it takes place.
- Customer accounts: Registered users have a profile containing their name, email, phone number, and a hashed password. Each user can make multiple bookings.
- Booking management: A booking is created when a user reserves one or more seats for a specific show. It records the booking date, overall status (e.g. confirmed, cancelled), and total price. A junction table links individual seats to bookings. Each completed booking is settled through a payment transaction that records the transaction ID, date, and payment status.

The system is designed to answer key operational queries such as: which seats are available for a given show, what bookings has a user made, how many seats remain in a hall for tonight's screening.

2. Entity-Relationship Diagram (ERD)

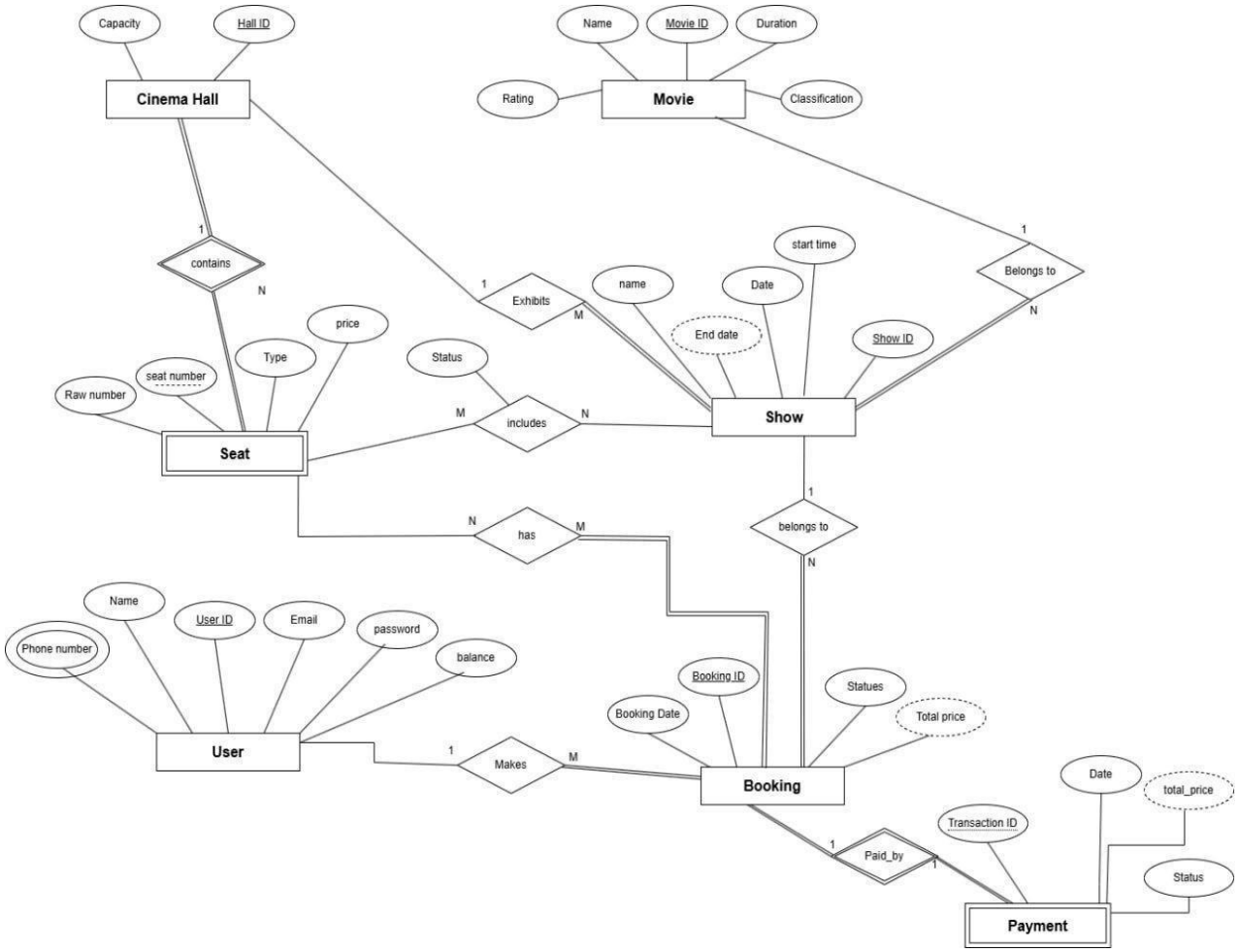
1.1 Entities and Attributes

The ERD identifies seven core entities. The primary key of each entity is underlined.

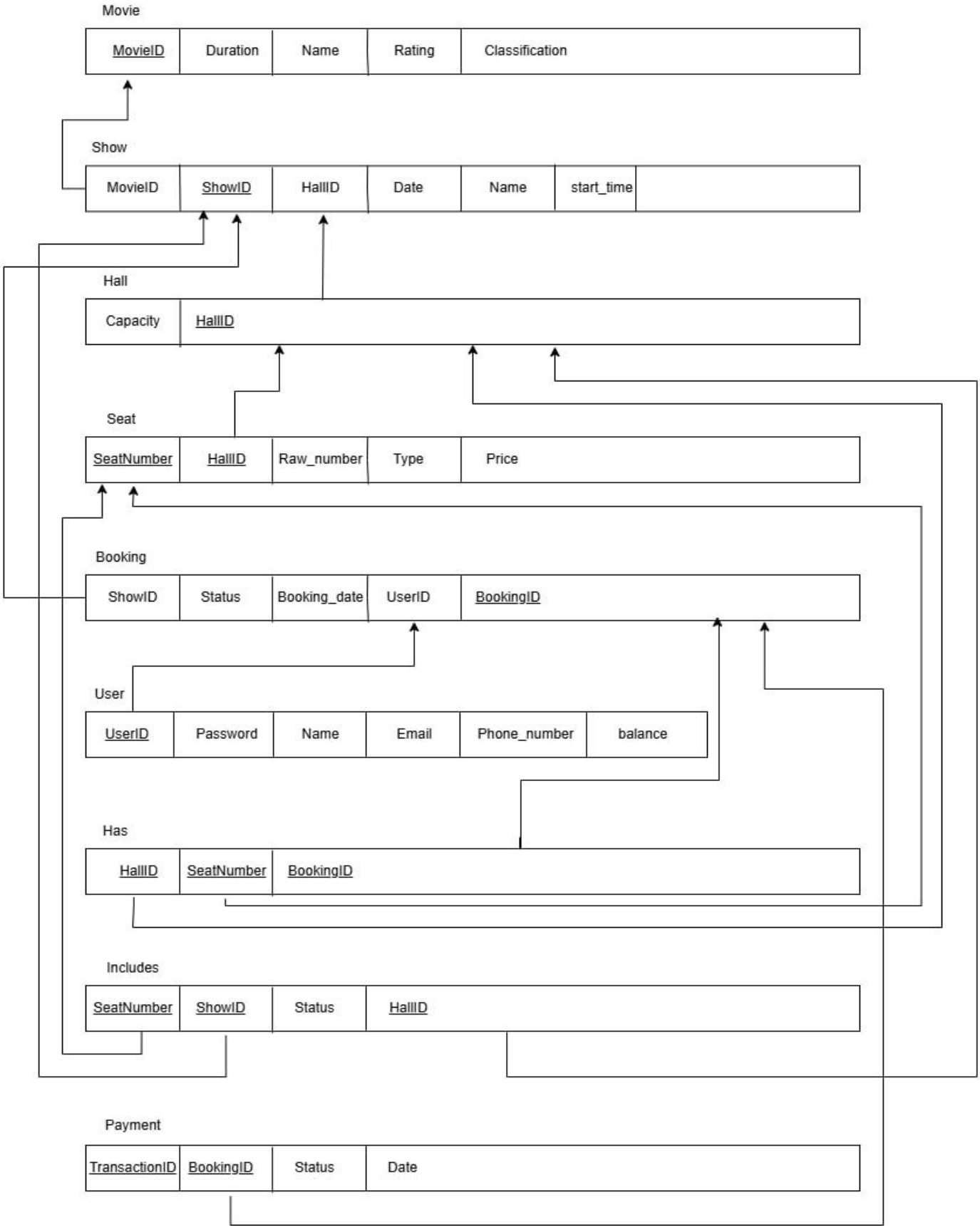
Entity	Primary Key	Attributes
Cinema Hall	HallID	Capacity
Movie	MovieID	Name, Duration, Rating, Classification
Show	ShowID	Name, Date, Start_time, Price
Seat	SeatNumber + HallID	Row_number, Type, Status
User	UserID	Name, Email, Phone_number, Password
Booking	BookingID	Booking_date, Status, Total_price
Payment	TransactionID	BookingID (FK), Status, Date

1.2 Relationships

Relationship	Cardinality	Description
Hall contains Seat	1 : N	One hall contains many seats; each seat belongs to exactly one hall.
Movie belongs to Show	1 : N	One movie can be scheduled for many shows; each show screens exactly one movie.
Hall exhibits Show	1 : N	One hall can host many shows; each show takes place in one hall.
User makes Booking	1 : N	One user can make many bookings; each booking belongs to one user.
Show belongs to Booking	1 : N	One show can have many bookings; each booking is for one show.
Booking reserves Seat	M : N	A booking can include multiple seats; a seat can appear in multiple bookings.
Booking paid by Payment	1 : 1	Each booking is settled by exactly one payment; each payment corresponds to one booking.


Capacity
Hall ID
Cinema Hall
Name
Movie ID
Duration
Rating
Movie
Classification
contains
Exhibits
name
Date
start time
Belongs to
End date
Show ID
price
Type
Status
seat number
Raw number
includes
Show
Seat
has
belongs to
Name
User ID
Email
password
Phone number
balance
Booking ID
Statues
Booking Date
Total price
User
Makes
Booking
Date
total_price
Transaction ID
Paid_by
Status
Payment

3. Relational Schema



4. First Normal Form (1NF)

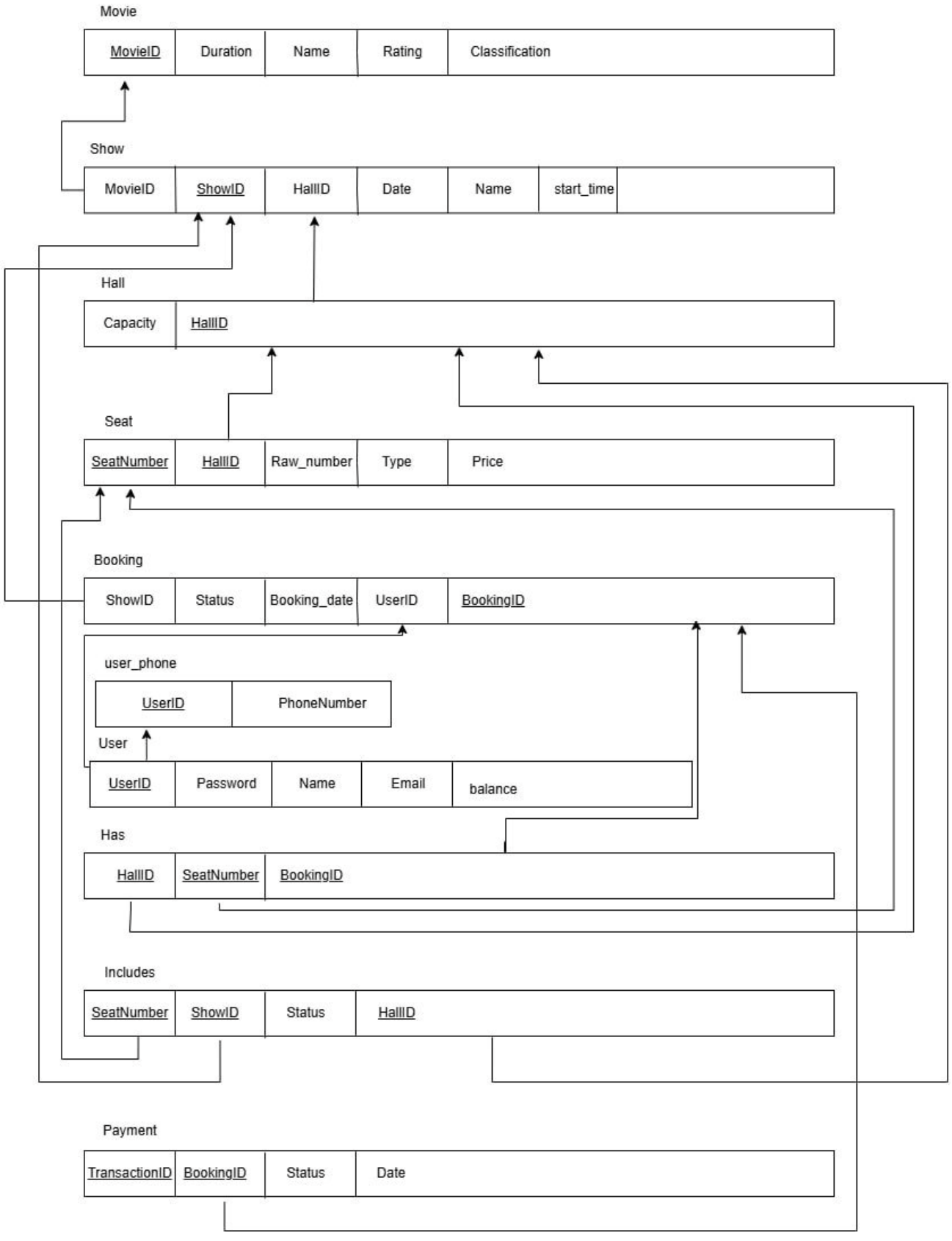
Normalization of User Contact Data:

- Converted the multi-valued PhoneNumber attribute into a dedicated USER_PHONE relation.
- Implemented a composite primary key (UserID, PhoneNumber) to allow multiple unique entries per user while maintaining data integrity.

Enforcement of Data Atomicity:

- Conducted a system-wide audit to ensure all attributes follow First Normal Form (1NF).
- Verified that every field contains a single, indivisible value, successfully removing all lists or grouped data from the schema.

Here is the schema that is in 1NF



5. Second Normal Form (2NF)

Only tables with composite primary keys can have partial dependencies. In the cinema schema these are: Seat, Has, Includes, and Payment.

Seat (PK: SeatNumber + HallID)

- Row_number: depends on SeatNumber + HallID together (a row number only makes sense relative to both a seat and its hall). Fully dependent. No violation.
- Type (standard/VIP): is a physical property of the seat within its hall. Fully dependent on the composite key. No violation.
- Status: represents current availability depends on the seat within its hall for a specific show so no violations.

HAS (PK: BookingID + SeatNumber + HallID)

This junction table has no non-key attributes, so there is nothing that could partially depend on a subset of the key. It is trivially in 2NF.

Includes (PK: SeatNumber + ShowID + HallID)

This junction table links seats to shows and carries a Status attribute (availability of that seat for that show). Status depends on the full composite key (SeatNumber + ShowID + HallID) together, since the availability of a seat is specific to a particular show in a particular hall. There is no partial dependency. The table is in 2NF.

Payment (PK: TransactionID)

Payment has a single-column primary key (TransactionID), so 2NF partial-dependency rules do not apply. Its non-key attributes (BookingID, Status, Date) all depend directly and fully on TransactionID. The table is in 2NF.

6. Third Normal Form (3NF)

Movie

- **Primary Key:** MovieID
- **Attributes:** Duration, Name, Rating, Classification
- **Analysis:** There is no transitive dependency here.

Booking

- **Primary Key:** BookingID
- **Attributes:** BookingDate, Status, Total_Price, UserID, ShowID
- **Analysis: There is no transitive dependency.**

Show (Potential Issue Found)

- **Primary Key:** ShowID
- **Attributes:** MovieID, Date, Start_Time, Price, Name, HallID
- **The Problem:** Look at the Name attribute.
- The Name refers to the Movie Name, it is already associated with MovieID.
- Since MovieID is a foreign key here (not the primary key), having Name in this table creates a transitive dependency:

  ShowID → MovieID → Name

User

- **Primary Key:** UserID
- **Attributes:** Name, Email, Password.
- **Analysis:** Each attribute depends directly on UserID. There is no relationship between Name and Email that would cause a shortcut. **No Transitive Dependencies**.

User_phone

- **Primary Key:** UserID, PhoneNumber.
- **Analysis: No Transitive Dependencies.**

Hall

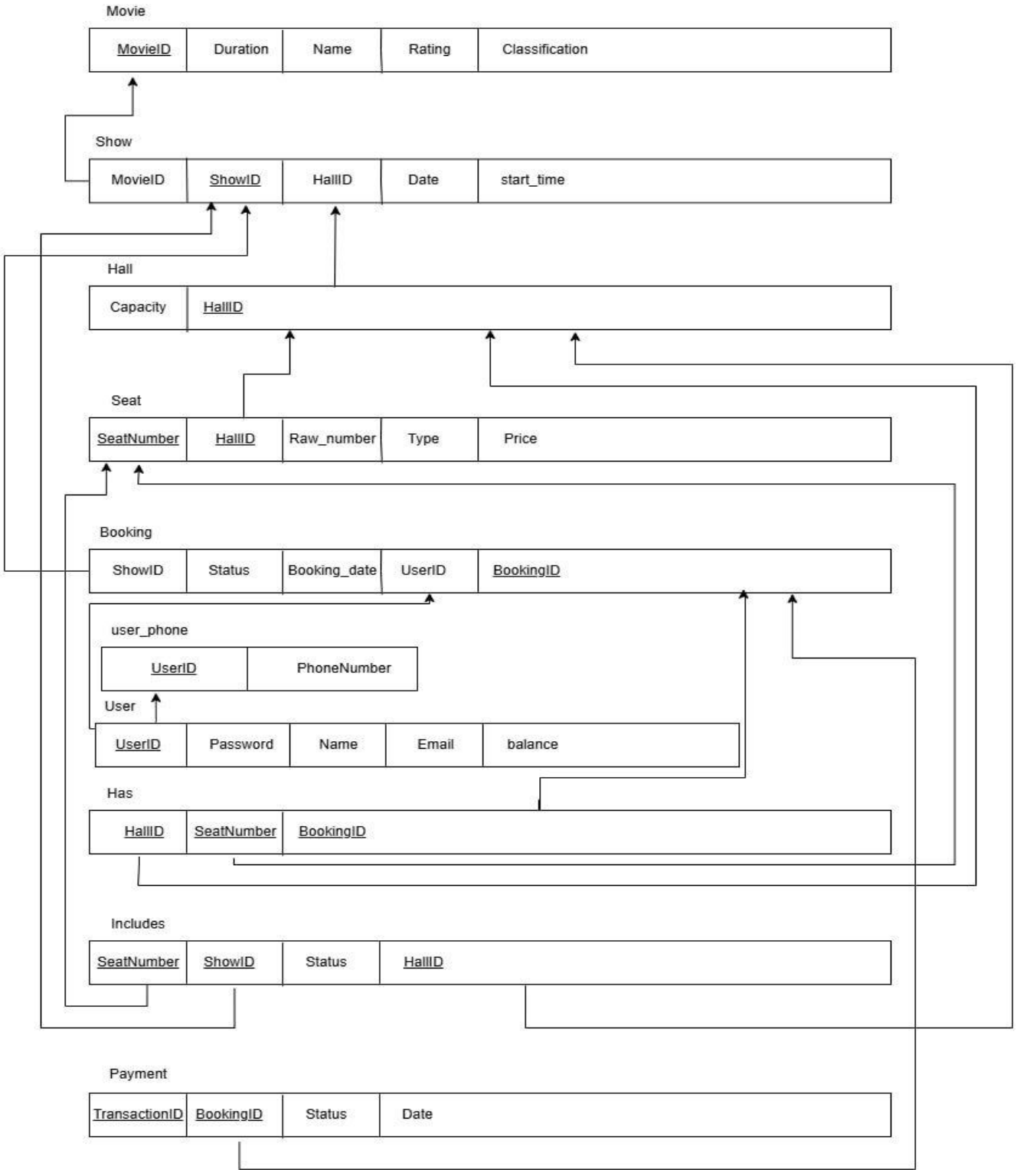
- **Primary Key:** HallID.
- **Attributes:** Capacity.
- **Analysis:** Capacity is a property of the hall itself. **No Transitive Dependencies.**

Seat

- **Primary Key:** SeatNumber, HallID.
- **Attributes:** Row_Number, Type.
- **Analysis:** The Type and Row_Number are properties of that specific seat in that specific hall. **No Transitive Dependencies.**

Show_seat, Has & Payment

- **Columns:** SeatNumber, ShowID, Status, HallID (Includes) / BookingID (Has) / TransactionID, BookingID, Status, Date (Payment).
- **Analysis:** These tables only contain keys and the immediate status of a specific reservation, seat state, or transaction. Payment attributes (Status, Date) depend directly on TransactionID with no transitive chain.
- **Verdict: No Transitive Dependencies.**


Movie
MovieID
Duration
Name
Rating
Classification
Show
MovieID
ShowID
HallID
Date
start_time
Hall
Capacity
HallID
Seat
SeatNumber
HallID
Raw_number
Type
Price
Booking
ShowID
Status
Booking_date
UserID
BookingID
user_phone
UserID
PhoneNumber
User
UserID
Password
Name
Email
balance
Has
HallID
SeatNumber
BookingID
Includes
SeatNumber
ShowID
Status
HallID
Payment
TransactionID
BookingID
Status
Date

7. Boyce-Codd Normal Form (BCNF)

7.1 Database Schema

The following diagram shows the final relational schema as implemented in the database, comprising all tables after normalization:

7.2 Functional Dependencies and BCNF Check

A table is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency X → Y, X is a superkey. All tables in the cinema schema satisfy this requirement. No violations were found and no decomposition is required.

Movie

- **Attributes:** MovieID, Name, Duration, Rating, Classification.
- **Candidate Key:** MovieID.
- **Functional Dependencies:** MovieID → (Name, Duration, Rating, Classification).
- **Verdict:** MovieID is a superkey. → In BCNF.

Hall

- **Attributes:** HallID, Capacity.
- **Candidate Key:** HallID.
- **Functional Dependencies:** HallID → Capacity.
- **Verdict:** HallID is a superkey. → In BCNF.

Show

- **Attributes:** ShowID, MovieID, HallID, Date, start_time.
- **Candidate Key:** ShowID.
- **Functional Dependencies:** ShowID → (MovieID, HallID, Date, start_time).
- **Verdict:** ShowID is a superkey. → In BCNF.

Seat

- **Attributes:** SeatNumber, HallID, Raw_number, Type, Price.
- **Candidate Key:** (SeatNumber, HallID).
- **Functional Dependencies:** (SeatNumber, HallID) → (Raw_number, Type, Price).
- **Verdict:** The composite key is the only determinant. → In BCNF.

User

- **Attributes:** UserID, Name, Email, Password, balance.
- **Candidate Key:** UserID.
- **Functional Dependencies:** UserID → (Name, Email, Password, balance).
- **Verdict:** UserID is a superkey. → In BCNF.

User_phone

- **Attributes:** UserID, PhoneNumber.
- **Candidate Key:** (UserID, PhoneNumber) — composite key, no non-key attributes.
- **Verdict:** No non-trivial FDs exist beyond the key itself. → Trivially in BCNF.

Booking

- **Attributes:** BookingID, ShowID, UserID, Status, Booking_date.
- **Candidate Key:** BookingID.
- **Functional Dependencies:** BookingID → (ShowID, UserID, Status, Booking_date).
- **Verdict:** BookingID is a superkey. → In BCNF.

Includes

- **Attributes:** SeatNumber, HallID, ShowID, Status.
- **Candidate Key:** (SeatNumber, HallID, ShowID).
- **Functional Dependencies:** (SeatNumber, HallID, ShowID) → Status.
- **Verdict:** The only determinant is the full composite key. → In BCNF.

Has

- **Attributes:** BookingID, SeatNumber, HallID.
- **Candidate Key:** (BookingID, SeatNumber, HallID) — all columns are key, no non-key attributes.
- **Verdict:** No non-trivial FDs exist beyond the key itself. → Trivially in BCNF.

Payment

- **Attributes:** TransactionID, BookingID, Status, Date.
- **Candidate Key:** TransactionID.
- **Functional Dependencies:** TransactionID → (BookingID, Status, Date).
- **Verdict:** TransactionID is a superkey. → In BCNF.

7.3 BCNF vs 3NF: Key Difference

BCNF is a stricter form of 3NF. A table is in BCNF if every determinant is a candidate key (superkey). 3NF allows a non-key attribute to be part of a candidate key as an exception; BCNF does not. In practice, this difference only surfaces when a table has two or more overlapping candidate keys. In the cinema schema, no such overlap exists in any table, so 3NF and BCNF are equivalent here — every table that satisfies 3NF also satisfies BCNF.

7.4 BCNF Status of the Cinema Schema

All ten tables (Movie, Hall, Show, Seat, User, user_phone, Booking, Includes, Has, Payment) satisfy BCNF. The schema required no decomposition at the BCNF stage. The final BCNF schema is therefore identical to the 3NF schema.

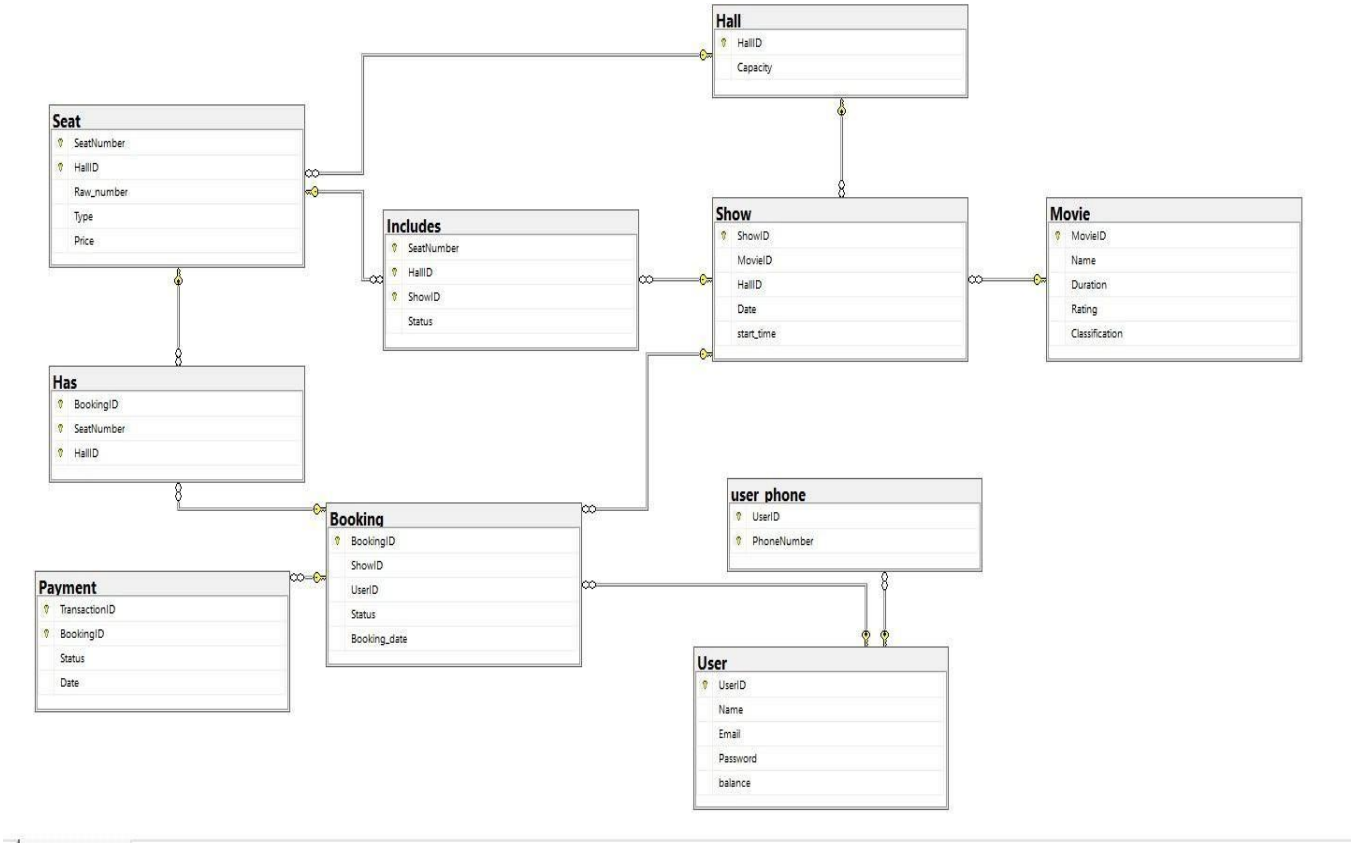
8. Normalization Summary

All eight tables in the cinema ticket system schema satisfy 1NF, 2NF, 3NF, and BCNF. The schema is well-designed with no normalization violations. The key design decisions that achieve this are:

- Using surrogate primary keys (MovieID, ShowID, BookingID, UserID, HallID) to avoid reliance on natural keys that could change.
- Resolving the many-to-many Seat-Booking relationship with the HAS table, eliminating repeating groups and ensuring atomicity.
- Storing all seat properties (Type, Row_number) in the Seat table and all booking properties (Booking_date, Status) in the Booking table, preventing partial dependencies from ever arising in the junction table.
- Ensuring no non-key attribute in any table depends on another non-key attribute, satisfying 3NF and BCNF throughout.

9. SQL Script

The following SQL scripts implement the Cinema Ticket System database on Microsoft SQL Server. They are 18rganized into five subsections: table creation, sample data insertion, user-defined functions, views, and stored procedures.



9.1 Table Creation

The script below creates all ten tables with their primary keys, foreign keys, and constraints, reflecting the final normalized schema.

```
CREATE TABLE Movie (
    MovieID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(255) NOT NULL,
    Duration INT,
    Rating VARCHAR(10),
    Classification VARCHAR(100)
);

CREATE TABLE Hall (
    HallID INT PRIMARY KEY IDENTITY(1,1),
    Capacity INT
);

CREATE TABLE [User] (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(255) NOT NULL,
    Email VARCHAR(255) UNIQUE,
    Password VARCHAR(255),
    balance DECIMAL(10, 2) DEFAULT 0.00
);


CREATE TABLE Show (
    ShowID INT PRIMARY KEY IDENTITY(1,1),
    MovieID INT,
    HallID INT,
    Date DATE,
    start_time TIME,
    CONSTRAINT FK_Show_Movie FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
    CONSTRAINT FK_Show_Hall FOREIGN KEY (HallID) REFERENCES Hall(HallID)
);


CREATE TABLE Seat (
    SeatNumber INT ,
    HallID INT ,
    Raw_number INT,
    Type VARCHAR(50),
    Price DECIMAL(10, 2),
    constraint seat_pk primary key(seatNumber, HallID),
    CONSTRAINT FK_Seat_Hall FOREIGN KEY (HallID) REFERENCES Hall(HallID)
);


CREATE TABLE user_phone (
    UserID INT,
    PhoneNumber VARCHAR(20),
    PRIMARY KEY (UserID, PhoneNumber),
    CONSTRAINT FK_Phone_User FOREIGN KEY (UserID) REFERENCES [User](UserID)
);


CREATE TABLE Booking (
    BookingID INT PRIMARY KEY IDENTITY(1,1),
    ShowID INT,
    UserID INT,
    Status VARCHAR(50),
    Booking_date DATETIME,
    CONSTRAINT FK_Booking_Show FOREIGN KEY (ShowID) REFERENCES Show(ShowID),
```

```
     CONSTRAINT FK_Booking_User FOREIGN KEY (UserID) REFERENCES [User](UserID)
);


CREATE TABLE Has (
    BookingID INT,
    SeatNumber INT,
    HallID INT,
    PRIMARY KEY (BookingID, SeatNumber, HallID),
    CONSTRAINT FK_Has_Booking FOREIGN KEY (BookingID) REFERENCES
Booking(BookingID),
    CONSTRAINT FK_Has_Seat FOREIGN KEY (SeatNumber, HallID) REFERENCES
Seat(SeatNumber, HallID)
);



CREATE TABLE Includes (
    SeatNumber INT,
    HallID INT,
    ShowID INT,
    Status VARCHAR(50),
    PRIMARY KEY (SeatNumber, HallID, ShowID),
    CONSTRAINT FK_Includes_Seat FOREIGN KEY (SeatNumber, HallID) REFERENCES
Seat(SeatNumber, HallID),
    CONSTRAINT FK_Includes_Show FOREIGN KEY (ShowID) REFERENCES Show(ShowID)
);


CREATE TABLE Payment (
    TransactionID INT IDENTITY(1,1),
    BookingID INT,
    PRIMARY key(TransactionID, BookingID ),
    Status VARCHAR(50),
    Date DATETIME,
    CONSTRAINT FK_Payment_Booking FOREIGN KEY (BookingID) REFERENCES
Booking(BookingID)
);
```

9.2 Sample Data

The following INSERT statements populate the database with representative test data covering movies, halls, users, shows, seats, bookings, and payments.

```
INSERT INTO Movie (Duration, Name, Rating, Classification) VALUES
(148, 'Inception', '8.8', 'Sci-Fi'),
(169, 'Interstellar', '8.7', 'Sci-Fi'),
(152, 'The Dark Knight', '9.0', 'Action'),
(121, 'The Martian', '8.0', 'Adventure');


INSERT INTO Hall (Capacity) VALUES
(50), (100), (150), (60);


INSERT INTO [User] (Name, Email, Password, balance) VALUES
('Ahmed Ali', 'ahmed@mail.com', 'p1', 500.00),
('Seif Shehta', 'seif@mail.com', 'p2', 200.00),
('Mona Zaki', 'mona@mail.com', 'p3', 1000.00),
('Omar Khalid', 'omar@mail.com', 'p4', 50.00),
('Sara Hassan', 'sara@mail.com', 'p5', 300.00),
('Youssef Adam', 'youssef@mail.com', 'p6', 400.00),
('Ziad Amr', 'ziad@mail.com', 'p7', 150.00);


INSERT INTO user_phone (UserID, PhoneNumber) VALUES
(1, '01011111111'), (1, '01222222222'),
(2, '01133333333'), (3, '01544444444'),
(3, '01099999999'), (4, '01055555555');


INSERT INTO Show (MovieID, Date, start_time, HallID) VALUES
(1, '2026-05-01', '13:00:00', 1),
(2, '2026-05-01', '20:00:00', 1),
(3, '2026-05-01', '18:00:00', 2),
(4, '2026-05-02', '15:00:00', 3);


INSERT INTO Seat (SeatNumber, HallID, Raw_number, Type, Price) VALUES
(1, 1, 1, 'Regular', 100.00),
(2, 1, 1, 'Regular', 100.00),
(3, 1, 1, 'Regular', 100.00),
(4, 1, 1, 'Regular', 100.00),
(1, 2, 1, 'VIP', 150.00),
(2, 2, 1, 'VIP', 150.00),
(10, 3, 2, 'Regular', 120.00);


INSERT INTO Booking (Booking_date, Status, UserID, ShowID) VALUES
('2026-04-15 10:30:00', 'Confirmed', 1, 1),
('2026-04-18 14:15:00', 'Confirmed', 2, 1),
('2026-04-20 09:00:00', 'Confirmed', 3, 2);


INSERT INTO Has (BookingID, SeatNumber, HallID) VALUES
(1, 1, 1), (1, 2, 1), (1, 3, 1),
(2, 4, 1), (3, 1, 1);


INSERT INTO Includes (SeatNumber, HallID, ShowID, Status) VALUES
(1, 1, 1, 'Reserved'), (2, 1, 1, 'Reserved'), (3, 1, 1, 'Reserved'),
(4, 1, 1, 'Reserved'), (1, 1, 2, 'Reserved');
```

```
INSERT INTO Payment (BookingID, Status, Date) VALUES
(1, 'Success', '2026-04-15 11:00:00'),
(2, 'Success', '2026-04-18 16:30:00'),
(3, 'Success', '2026-04-20 09:15:00');
```

9.3 User-Defined Functions

9.3.1 fn_CalculateTotalPrice

Calculates the total price for a booking by summing seat prices, with an optional discount percentage parameter.

```
CREATE FUNCTION fn_CalculateTotalPrice(
    @bookingId INT,
    @discountPercent DECIMAL(5,2) = 0
)
RETURNS MONEY
AS
BEGIN
    DECLARE @subtotal MONEY;

    SELECT @subtotal = SUM(S.Price)
    FROM Has H
    JOIN Seat S ON H.SeatNumber = S.SeatNumber AND H.HallID = S.HallID
    WHERE H.BookingID = @bookingId;

    RETURN ISNULL(@subtotal, 0) * (1 - @discountPercent / 100.0);
END;
```

9.3.2 checkSeatAvailability

Returns the availability status ('Available', 'Reserved', or 'Not Found') of a specific seat for a given show.

```
CREATE FUNCTION checkSeatAvailability(
    @seatNumber INT,
    @hallid INT,
    @showid INT
)
RETURNS VARCHAR(50)
AS
BEGIN
    DECLARE @seatStatus VARCHAR(50);

    SELECT @seatStatus = Status
    FROM Includes
    WHERE SeatNumber = @seatNumber
      AND HallID = @hallid
      AND ShowID = @showid;

    RETURN ISNULL(@seatStatus, 'Not Found');
END;
GO
```

9.4 Views

9.4.1 vw_CustomerBookings

A reporting view that joins all relevant tables to show each booking with customer name, movie, hall, seat breakdown by type, total price, and booking status.

```
CREATE OR ALTER VIEW vw_CustomerBookings AS
SELECT
    U.Name AS [Customer],
    M.Name AS [Movie Name],
    S.HallID AS [Hall ID],
    dbo.fn_CalculateTotalPrice(B.BookingID, 0) AS [Total Price],
    B.Status AS [Status],
    B.Booking_date AS [Date],
    STRING_AGG(CAST(ST.SeatNumber AS VARCHAR), ', ') AS [Seats List],
    COUNT(CASE WHEN ST.Type = 'Regular' THEN 1 END) AS [Regular Seats],
    COUNT(CASE WHEN ST.Type = 'VIP' THEN 1 END) AS [VIP Seats],
    COUNT(CASE WHEN ST.Type = 'Premium' THEN 1 END) AS [Premium Seats]
FROM Booking B
JOIN [User] U ON B.UserID = U.UserID
JOIN Show S ON B.ShowID = S.ShowID
JOIN Movie M ON S.MovieID = M.MovieID
JOIN Has H ON B.BookingID = H.BookingID
JOIN Seat ST ON H.SeatNumber = ST.SeatNumber AND H.HallID = ST.HallID
GROUP BY
    B.BookingID,
    U.Name,
    M.Name,
    S.HallID,
    B.Status,
    B.Booking_date;
GO
```

9.4.2 vw_AvailableSeats

A view that lists all available seats for every show by checking the Includes table for 'Available' status or absence of a record.

```

CREATE OR ALTER VIEW vw_AvailableSeats AS
SELECT
    s.ShowID,
    m.Name AS MovieName,
    s.Date AS ShowDate,
    s.start_time,
    h.HallID,
    h.Capacity,
    st.SeatNumber,
    st.Raw_number,
    st.Type,
    st.Price
FROM Show s
JOIN Movie m ON s.MovieID = m.MovieID
JOIN Hall h ON s.HallID = h.HallID
JOIN Seat st ON h.HallID = st.HallID
LEFT JOIN Includes i ON st.SeatNumber = i.SeatNumber
                    AND st.HallID = i.HallID
                    AND s.ShowID = i.ShowID
WHERE i.Status IS NULL OR i.Status = 'Available';
GO
```

9.4.3 vw_BookingDetail

Provides a comprehensive overview of each booking, including customer details, show information, movie data, and payment status.

```
CREATE OR ALTER VIEW vw_BookingDetails AS
SELECT
b.BookingID,
b.Booking_date,
b.Status AS BookingStatus,
u.UserID,
u.Name AS CustomerName,
u.Email,
s.ShowID,
s.Date AS ShowDate,
s.start_time,
m.MovieID,
m.Name AS MovieName,
m.Duration,
h.HallID,
h.Capacity,
p.Status AS PaymentStatus,
p.Date AS PaymentDate
FROM Booking b
JOIN [User] u ON b.UserID = u.UserID
JOIN Show s ON b.ShowID = s.ShowID
JOIN Movie m ON s.MovieID = m.MovieID
JOIN Hall h ON s.HallID = h.HallID
LEFT JOIN Payment p ON b.BookingID = p.BookingID;
```

9.4.4 vw_HallOccupancy

Calculates the occupancy statistics for each hall and show, including the number of booked seats, available seats, and the occupancy percentage.

```
CREATE OR ALTER VIEW vw_HallOccupancy AS
SELECT
h.HallID,
h.Capacity,
s.ShowID,
s.Date AS ShowDate,
s.start_time,
COUNT(DISTINCT hs.SeatNumber) AS BookedSeats,
h.Capacity - COUNT(DISTINCT hs.SeatNumber) AS AvailableSeats,
CAST(
COUNT(DISTINCT hs.SeatNumber) * 100.0 / NULLIF(h.Capacity, 0)
AS DECIMAL(5,2)
) AS OccupancyPercent
FROM Hall h
JOIN Show s ON h.HallID = s.HallID
LEFT JOIN Booking b ON s.ShowID = b.ShowID AND b.Status = 'Confirmed'
LEFT JOIN Has hs ON b.BookingID = hs.BookingID
GROUP BY h.HallID, h.Capacity, s.ShowID, s.Date, s.start_time;
```

9.4.5 vw_MovieRevenue

Summarizes the total bookings and total revenue generated by each movie based on confirmed bookings.

```
CREATE OR ALTER VIEW vw_MovieRevenue AS
SELECT
m.MovieID,
m.Name AS MovieName,
m.Classification,
COUNT(DISTINCT b.BookingID) AS TotalBookings,
ISNULL(SUM(dbo.fn_CalculateTotalPrice(b.BookingID, 0)), 0) AS
TotalRevenue
FROM Movie m
LEFT JOIN Show s ON m.MovieID = s.MovieID
LEFT JOIN Booking b ON s.ShowID = b.ShowID AND b.Status = 'Confirmed'
GROUP BY m.MovieID, m.Name, m.Classification;
```

9.5 Stored Procedures

9.5.1 RegisterUser

Registers a new user by inserting into the User and user_phone tables within a single transaction. Raises an error if the email already exists.

```
CREATE PROCEDURE [dbo].[RegisterUser]
    @name VARCHAR(255),
    @email VARCHAR(255),
    @password VARCHAR(255),
    @phoneNumber VARCHAR(20),
    @balance DECIMAL(10, 2) -- The new 5th parameter
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Check if the Email already exists in the database
    IF EXISTS (
        SELECT 1 FROM [User]
        WHERE Email = @email
    )
    BEGIN
        RAISERROR('Email already exists.', 16, 1);
        RETURN;
    END

    DECLARE @userId INT;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 2. Insert into the main [User] table using the balance from the form
        INSERT INTO [User] (Name, Email, Password, balance)
        VALUES (@name, @email, @password, @balance);

        -- Get the ID of the user we just created
        SET @userId = SCOPE_IDENTITY();

        -- 3. Insert into the separate phone table using the new UserID
        INSERT INTO user_phone (UserID, PhoneNumber)
        VALUES (@userId, @phoneNumber);
```

```
        COMMIT TRANSACTION;
        PRINT 'User registered successfully with UserID: ' + CAST(@userId AS
VARCHAR);

    END TRY
    BEGIN CATCH
        -- If any step fails, undo everything to keep data consistent
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
GO
```

9.5.2 BOOK_TICKET

Books one or more seats for a show. Creates a Booking record, links seats via the Has table, marks them as Reserved in the Includes table, and triggers balance payment — all within one atomic transaction.

```
   CREATE TYPE [dbo].[SeatTableType] AS TABLE(
        seatNumber INT
    );

CREATE PROCEDURE [dbo].[BOOK_TICKET]
    @userId INT,
    @showId INT,
    @Seats SeatTableType READONLY
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @hallId INT;
        DECLARE @bookingId INT;

        SELECT @hallId = HallID FROM Show WHERE ShowID = @showId;
        IF @hallId IS NULL THROW 50002, 'Show does not exist.', 1;

        INSERT INTO Booking (ShowID, UserID, Status, Booking_date)
        VALUES (@showId, @userId, 'Pending', GETDATE());
        SET @bookingId = SCOPE_IDENTITY();

        INSERT INTO Has (BookingID, SeatNumber, HallID)
        SELECT @bookingId, seatNumber, @hallId FROM @Seats;

        UPDATE i SET Status = 'Reserved'
        FROM Includes i JOIN @Seats s ON i.SeatNumber = s.seatNumber
        WHERE i.ShowID = @showId AND i.HallID = @hallId;

        EXEC dbo.ProcessBalancePayment @bookingId = @bookingId, @userId =
@userId;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

9.5.3 CancelBooking

Cancels a single booking if it belongs to the requesting user and the show is more than 24 hours away. Frees the reserved seats, refunds the full price to the user's balance, and records a Refunded payment entry.

```
CREATE PROCEDURE [dbo].[CancelBooking]
    @bookingId INT,
    @userId INT,
    @refundAmount DECIMAL(10,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @showId INT;
    DECLARE @TotalPrice DECIMAL(10,2);
    DECLARE @ShowDateTime DATETIME;

    SET @TotalPrice = dbo.fn_CalculateTotalPrice(@bookingId, 0);

    SELECT
        @showId = B.ShowID,
        @ShowDateTime = CAST(S.Date AS DATETIME) + CAST(S.start_time AS
DATETIME)
    FROM Booking B
    INNER JOIN Show S ON S.ShowID = B.ShowID
    WHERE B.BookingID = @bookingId AND B.UserID = @userId;

    IF @showId IS NULL
    BEGIN
        RAISERROR('Booking not found or does not belong to this user.', 16, 1);
        RETURN;
    END

    IF EXISTS (SELECT 1 FROM Booking WHERE BookingID = @bookingId AND Status =
'Cancelled')
    BEGIN
        RAISERROR('Booking is already cancelled.', 16, 1);
        RETURN;
    END

    IF @ShowDateTime < DATEADD(HOUR, 24, GETDATE())
    BEGIN
        RAISERROR('Cannot cancel a booking within 24 hours of showtime.', 16,
1);
        RETURN;
    END

    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE I
        SET I.Status = 'Available'
        FROM Includes I
        INNER JOIN Has H ON I.SeatNumber = H.SeatNumber AND I.HallID = H.HallID
        WHERE I.ShowID = @showId AND H.BookingID = @bookingId;

        UPDATE Booking
        SET Status = 'Cancelled'
        WHERE BookingID = @bookingId;

        UPDATE [User]
        SET balance = balance + @TotalPrice
```

```
        WHERE UserID = @userId;

        INSERT INTO Payment (BookingID, Status, Date)
        VALUES (@bookingId, 'Refunded', GETDATE());

        SET @refundAmount = @TotalPrice;

        COMMIT TRANSACTION;
        PRINT 'Booking cancelled successfully. Amount ' + CAST(@TotalPrice AS
VARCHAR) + ' returned to your balance.';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

9.5.4 CancelShow

Cancels an entire show. Computes refunds for all non-cancelled bookings, credits each user's balance, inserts Refunded payment records, and marks all seats as Available.

```
CREATE PROCEDURE [dbo].[CancelShow]
    @showId INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Show WHERE ShowID = @showId)
    BEGIN
        RAISERROR('Invalid showId.', 16, 1);
        RETURN;
    END

    DECLARE @RefundList TABLE (
        bookingId INT,
        userId     INT,
        refundAmount DECIMAL(10,2)
    );

    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO @RefundList (bookingId, userId, refundAmount)
        SELECT B.BookingID, B.UserID, dbo.fn_CalculateTotalPrice(B.BookingID,
0)
        FROM Booking B
        WHERE B.ShowID = @showId AND B.Status <> 'Cancelled';

        UPDATE U
        SET U.balance = U.balance + R.refundAmount
        FROM [User] U
        INNER JOIN @RefundList R ON U.UserID = R.userId;

        INSERT INTO Payment (BookingID, Status, Date)
        SELECT bookingId, 'Refunded: Show Cancelled', GETDATE()
        FROM @RefundList;

        UPDATE Booking
        SET Status = 'Cancelled'
```

```
        WHERE ShowID = @showId;

        UPDATE Includes
        SET Status = 'Available'
        WHERE ShowID = @showId;

        SELECT bookingId, userId, refundAmount
        FROM @RefundList;

        COMMIT TRANSACTION;
        PRINT 'Show cancelled and all customers have been refunded
successfully.';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
GO
```

9.5.5 showAllAvailableSeatsForASpecificShow

Returns all seats that are currently Available for a given show by joining the Seat and Includes tables and calling checkSeatAvailability.

```
CREATE PROCEDURE [dbo].[showAllAvaliableSeatsForASpecificShow]
    @showid INT
AS
BEGIN
    SELECT
        S.SeatNumber,
        S.Raw_number,
        S.Type,
        S.HallID,
        I.ShowID
    FROM Seat S
    INNER JOIN Includes I ON S.SeatNumber = I.SeatNumber AND S.HallID =
I.HallID
    WHERE I.ShowID = @showid
      AND dbo.checkSeatAvailability(S.SeatNumber, S.HallID, I.ShowID) =
'Available';
END
GO
```

9.5.6 DeleteUser

Deletes a user and all associated records (payments, Has entries, bookings, phone numbers) in the correct dependency order, freeing any reserved seats before removal.

```
CREATE PROCEDURE [dbo].[DeleteUser]
    @userId INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM [User] WHERE UserID = @userId)
    BEGIN
        RAISERROR('INVALID USERID.', 16, 1);
        RETURN;
```

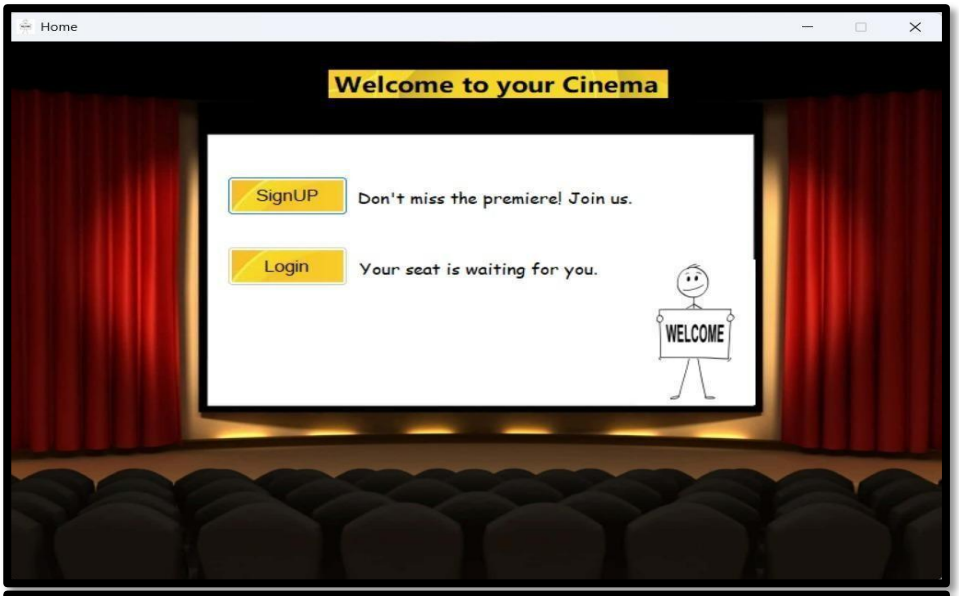
```
    END

    BEGIN TRY
        BEGIN TRANSACTION;
        UPDATE I
        SET I.Status = 'Available'
        FROM Includes I
        INNER JOIN Has H ON I.SeatNumber = H.SeatNumber AND I.HallID = H.HallID
        INNER JOIN Booking B ON H.BookingID = B.BookingID
        WHERE B.UserID = @userId AND B.Status <> 'Cancelled';

        DELETE FROM Payment
        WHERE BookingID IN (SELECT BookingID FROM Booking WHERE UserID =
@userId);

        DELETE FROM Has
        WHERE BookingID IN (SELECT BookingID FROM Booking WHERE UserID =
@userId);

        DELETE FROM Booking WHERE UserID = @userId;

        DELETE FROM user_phone WHERE UserID = @userId;

        DELETE FROM [User] WHERE UserID = @userId;

        COMMIT TRANSACTION;
        PRINT 'User and all related records (including freed seats) deleted
successfully.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
GO
```

9.5.6 ProcessBalancePayment

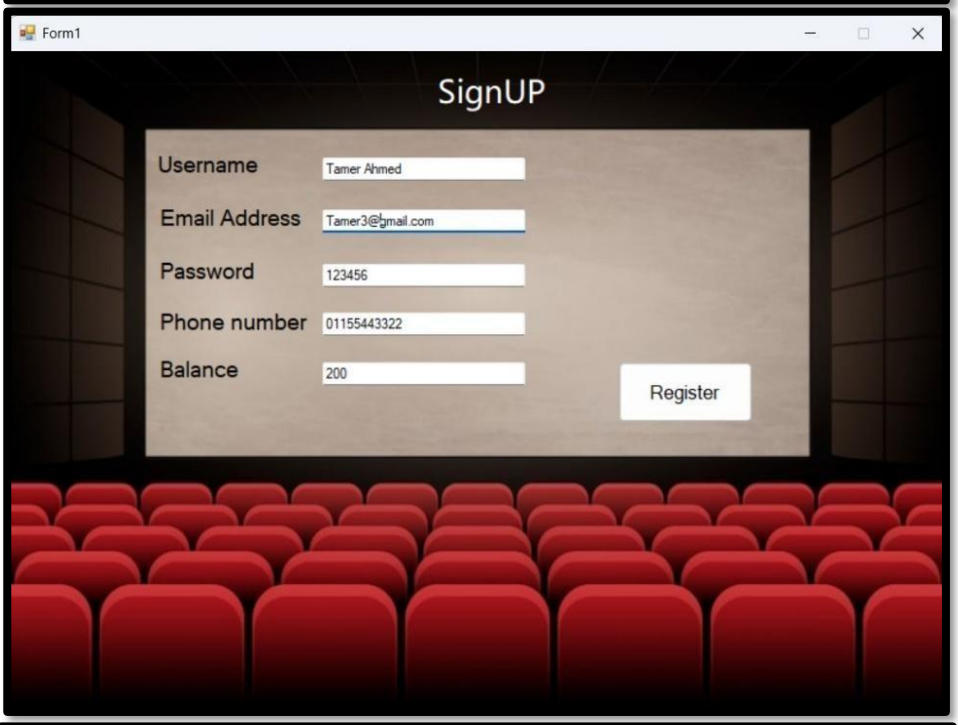
Processes the payment for a booking using the user's balance. If the balance is sufficient, the payment is recorded as successful, the booking is confirmed, and the seats are reserved.
Otherwise, the payment fails, the booking is cancelled, and the seats are released.

```
CREATE PROCEDURE ProcessBalancePayment
    @bookingId INT,
    @userId INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @totalPrice DECIMAL(10,2);
    SET @totalPrice = dbo.fn_CalculateTotalPrice(@bookingId, 0);

    UPDATE [User]
    SET balance = balance - @totalPrice
    WHERE UserID = @userId AND balance >= @totalPrice;

    IF @@ROWCOUNT > 0
    BEGIN
        INSERT INTO Payment (BookingID, Status, Date)
        VALUES (@bookingId, 'Success', GETDATE());

        UPDATE Booking SET Status = 'Confirmed' WHERE BookingID = @bookingId;

        UPDATE i
        SET i.Status = 'Reserved'
        FROM Includes i
        JOIN Has H ON i.SeatNumber = H.SeatNumber AND i.HallID = H.HallID
        JOIN Booking B ON H.BookingID = B.BookingID
        WHERE B.BookingID = @bookingId AND i.ShowID = B.ShowID;

        PRINT 'Payment Successful.';
    END
    ELSE
    BEGIN
        INSERT INTO Payment (BookingID, Status, Date)
        VALUES (@bookingId, 'Failed: Insufficient Funds', GETDATE());

        UPDATE Booking SET Status = 'Cancelled' WHERE BookingID = @bookingId;

        UPDATE i
        SET i.Status = 'Available'
        FROM Includes i
        JOIN Has H ON i.SeatNumber = H.SeatNumber AND i.HallID = H.HallID
        JOIN Booking B ON H.BookingID = B.BookingID
        WHERE B.BookingID = @bookingId AND i.ShowID = B.ShowID;

        PRINT 'Payment Failed: Insufficient balance. Seats have been
released.';
    END
   END;
```

10. GUI Screenshots:



Home Page

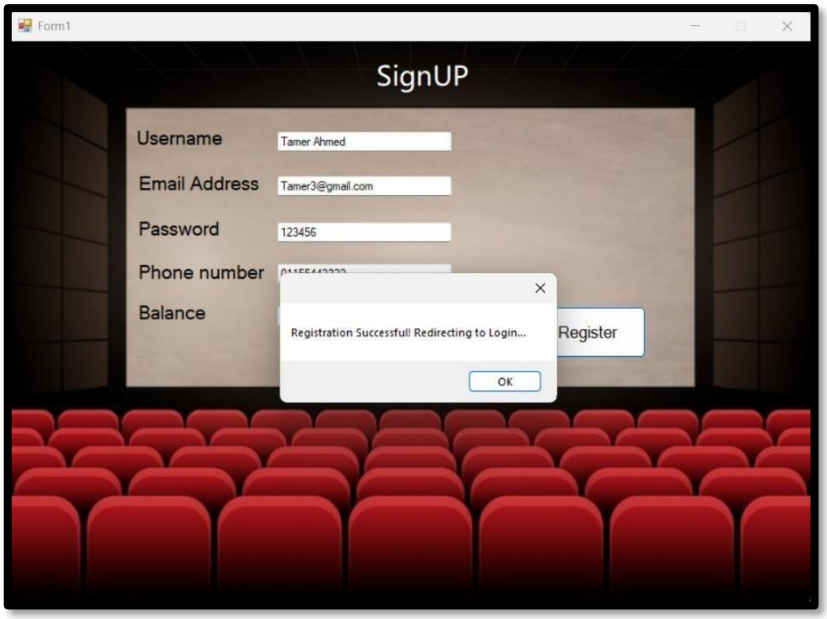


Registration Page

Results Messages

	UserID	Name	Email	Password	balance
1	1	Ahmed Ali	ahmed@mail.com	p1	500.00
2	2	Seif Shehta	seif@mail.com	p2	200.00
3	3	Mona Zaki	mona@mail.com	p3	1000.00
4	4	Omar Khalid	omar@mail.com	p4	50.00
5	5	Sara Hassan	sara@mail.com	p5	300.00
6	6	Youssef Adam	youssef@mail.com	p6	400.00
7	7	Ziad Amr	ziad@mail.com	p7	150.00

Users in the DB

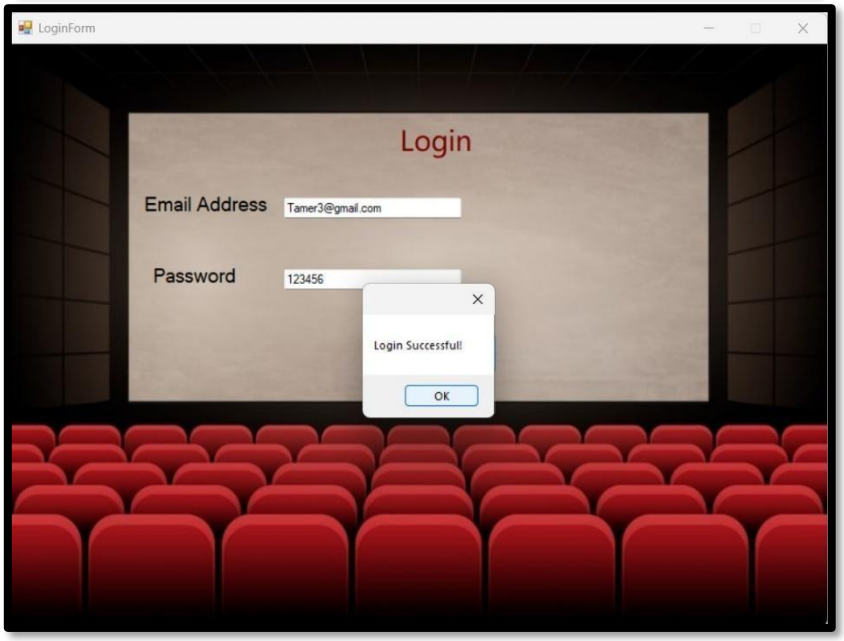


Registration success

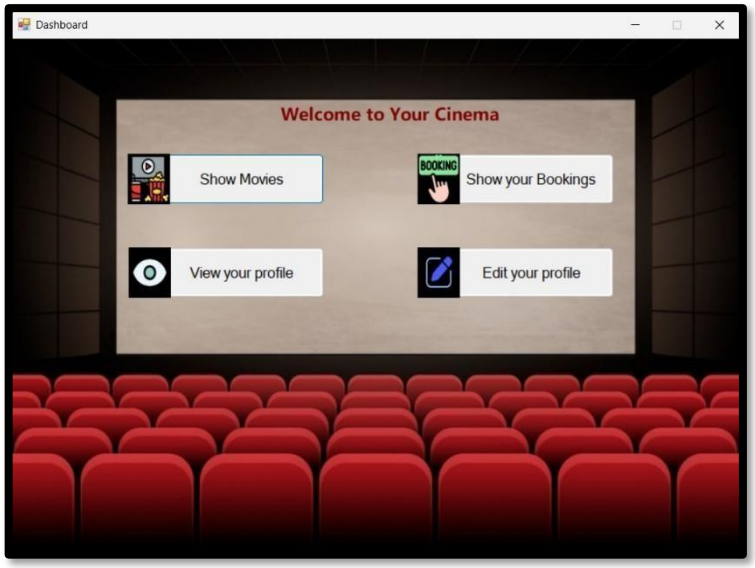
Results Messages

	UserID	Name	Email	Password	balance
1	1	Ahmed Ali	ahmed@mail.com	p1	500.00
2	2	Seif Shehta	seif@mail.com	p2	200.00
3	3	Mona Zaki	mona@mail.com	p3	1000.00
4	4	Omar Khalid	omar@mail.com	p4	50.00
5	5	Sara Hassan	sara@mail.com	p5	300.00
6	6	Youssef Adam	youssef@mail.com	p6	400.00
7	7	Ziad Amr	ziad@mail.com	p7	150.00
8	1002	Tamer Ahmed	Tamer3@gmail.com	123456	200.00

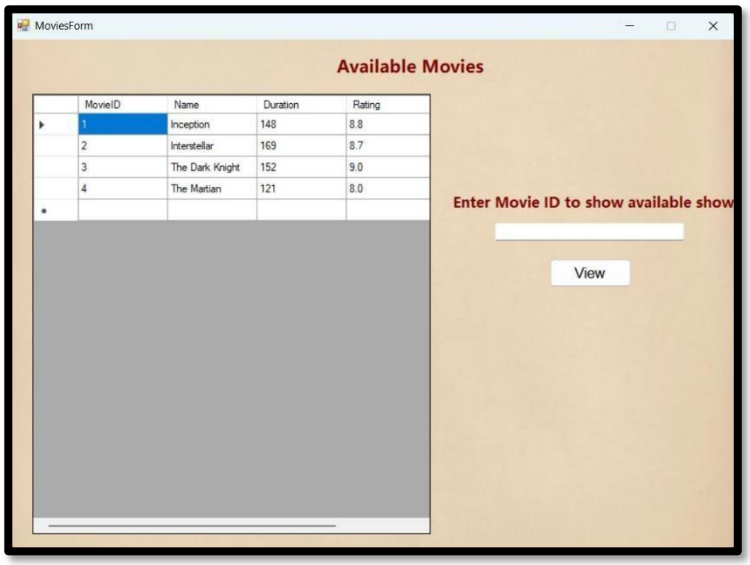
Users in the DB after register



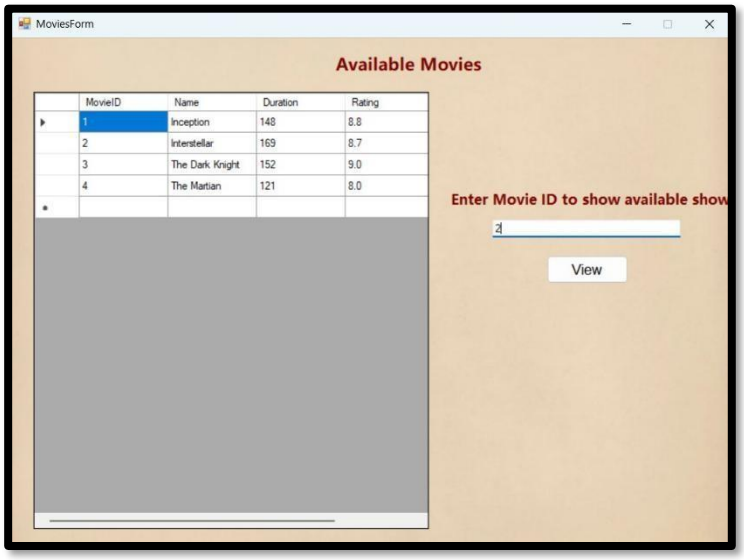
Login Page



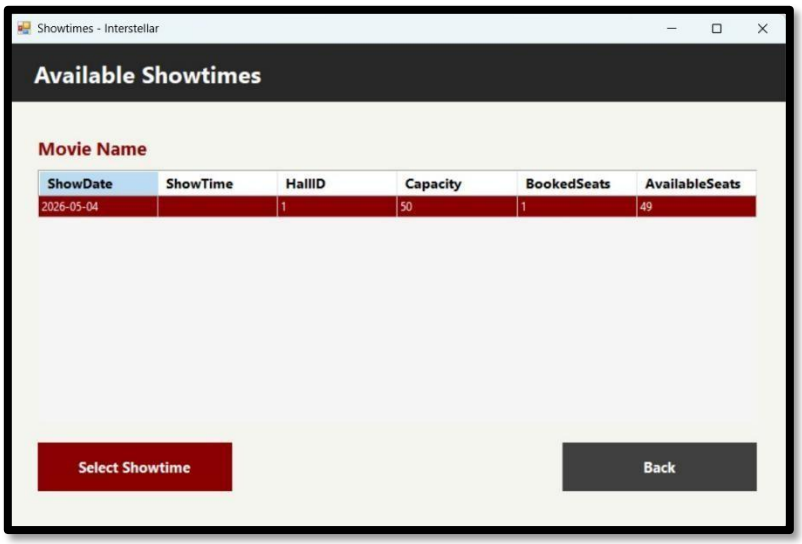
Main Dashboard



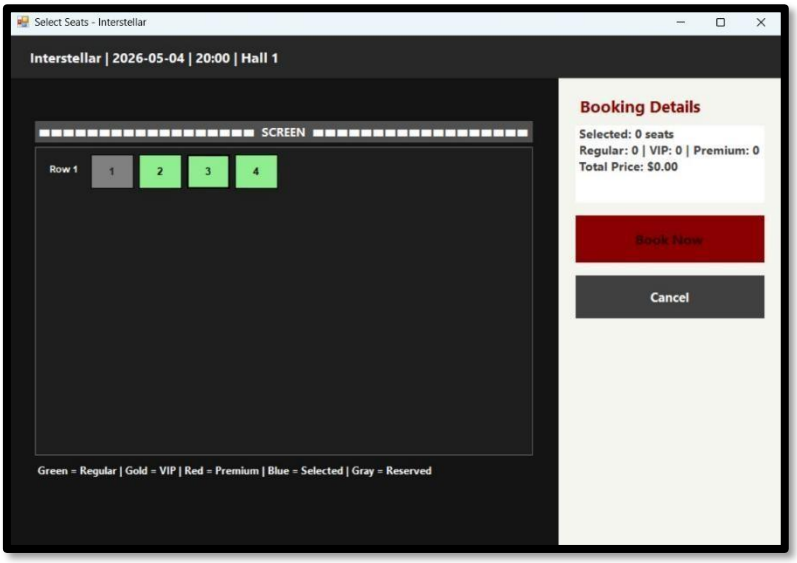
Available Movies form



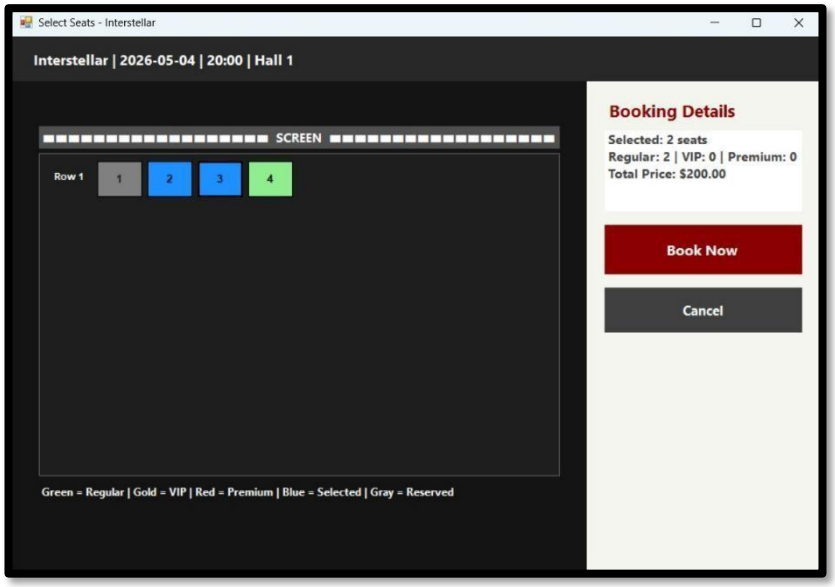
Enter a movie ID to see available shows



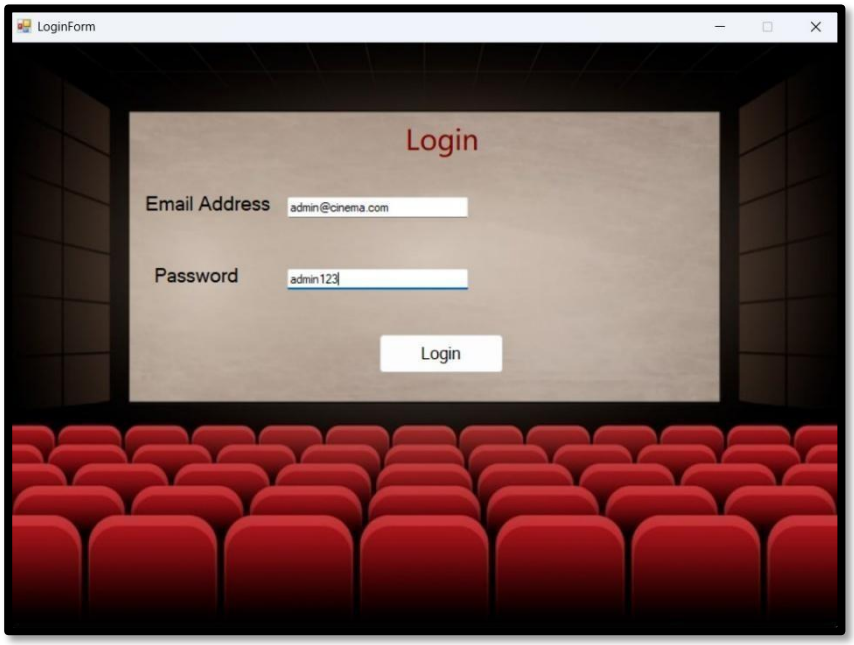
Available Shows



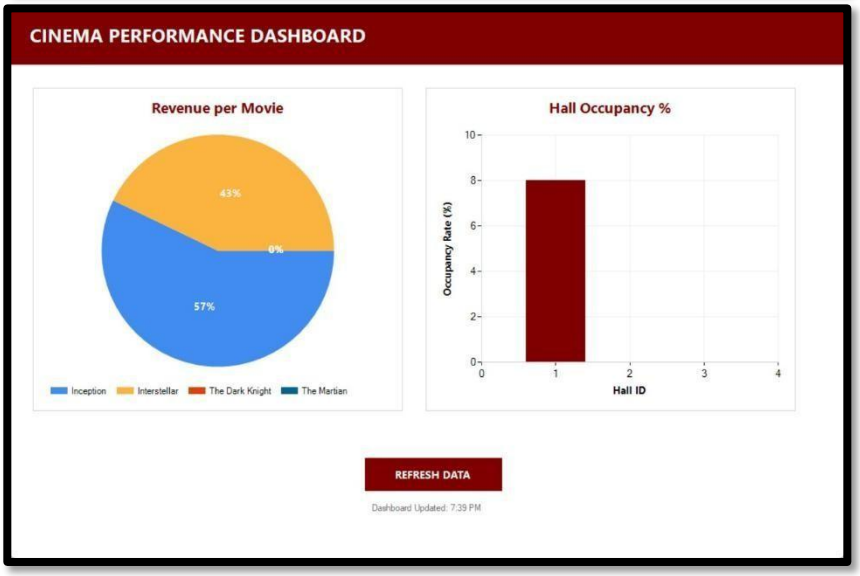
Available seats for the selected show



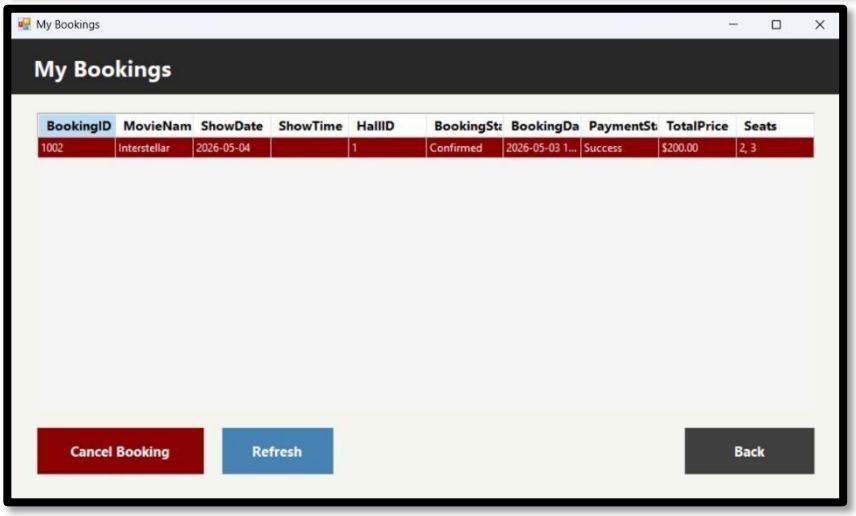
Make a new booking with seats #2,3



Login as an admin



Manager Dashboard



Show bookings

References

[1] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970. doi: 10.1145/362384.362685

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[3] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[5] E. F. Codd, “Further normalization of the data base relational model,” in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[6] R. F. Boyce and E. F. Codd, “Boyce–Codd normal form,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Ann Arbor, MI, USA, 1974, pp. 17–20.

[7] P. P. Chen, “The entity-relationship model—toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976. doi: 10.1145/320434.320440

[8] Microsoft Corporation, “Transact-SQL reference (Database Engine),” *Microsoft SQL Server Documentation*. Microsoft, Redmond, WA, USA. [Online]. Available: https://learn.microsoft.com/en-us/sql/t-sql/language-reference. [Accessed: May 2025].

[9] Microsoft Corporation, “Stored procedures (Database Engine),” *Microsoft SQL Server Documentation*. Microsoft, Redmond, WA, USA. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures. [Accessed: May 2025].

[10] Microsoft Corporation, “Views (SQL Server),” *Microsoft SQL Server Documentation*. Microsoft, Redmond, WA, USA. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/views. [Accessed: May 2025].

[11] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, UK: Pearson Education, 2015.

[12] J. L. Harrington, *Relational Database Design and Implementation*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[13] Ain Shams University, Faculty of Engineering, “CSE244: Database Systems Design — Course Notes and Lecture Slides,” Cairo, Egypt, 2024–2025.